Unit 7: Advance Topics on JavaScript

Presented By:
Bipin Maharjan

Learning Objectives

- Grasp the intricacies of scope, variable visibility, and the concept of closures
- Manage errors using try-catch blocks, exceptions, and employ debugging techniques
- Manipulate the Document Object Model (DOM) to interact with HTML elements and handle events
- Master asynchronous programming, including callback functions, promises, and async/await
- Work with JSON data and make AJAX requests using fetch API or XMLHttpRequest
- Explore ES6 features like arrow functions, template literals, let and const keywords, and destructuring assignments
- Understand modern JavaScript concepts, such as modules and import/export functionality
- Gain exposure to popular JavaScript libraries/frameworks like React, Angular, or Vue.js for building web applications
- Elevate JavaScript skills to enable dynamic and interactive web development

Table of Contents

- Scope and Closures
- Error Handling and Debugging
- DOM Manipulation
- Asynchronous JavaScript
- JSON and AJAX
- ES6 and Modern JavaScript
- JavaScript Libraries

Scope and Closures

What is scope?

- Scope means lifespan of a variables.
- It determine accessibility/visibility of the variables inside the JavaScript code.

There are two types of scope

- Global Scope
- Local Scope
  - Function Scope
  - Block Scope

Global Scope

- The variable which is declared in the global execution context is called Global Scope.
- Global Scope variable can accessible from anywhere within the code.

Example

```
var globalVar = "I'm global";

function showGlobal() {
    console.log(globalVar);   // Accessible
}
showGlobal();
console.log(globalVar);  // Accessible everywhere
```

Local Scope

- Locally scoped variables are only visible and accessible within their local scope.
- There are two types of local scope:
  - Function Scope
  - Block Scope

Function Scope

- The variable which is declared inside the function is called function scope.
- The function scope variable cannot be accessed or modified outside the function

Example

```
function func() {
	let a = "function scope";
	console.log(a);  //Accessible
}
func();
console.log(a);   // Error
```

Block Scope

- The variable which is declared inside the curly braces (if, for, while loops) called block scope.
- The block scope variable can be accessed only within the block of code not outside of the curly braces.
- “Let” and “const” keywords used to declare the block scope variable

Block Scope Example

```
function func() {
	if (true) {
		var a =5;
		let b = 10;
		const c = 20;
	}
	console.log(a); //5
	console.log(b); //ReferenceError: b is not defined
	console.log(c); //ReferenceError: c is not defined
}
```

What is Closure?

A closure is a function that remembers and accesses variables from its outer scope even after the outer function has finished executing.

- Retains access to outer function variables.
- Preserves the lexical scope.
- Allows data encapsulation and privacy.
- Commonly used in callbacks and asynchronous code.

Closure Example

```
function outerFunction() {
    let count = 0;

    function innerFunction() {
        count++;
        console.log(count);
    }

    return innerFunction;
}

let counter = outerFunction();
counter(); // 1
counter(); // 2
counter(); // 3
```

- outerFunction() executes
- innerFunction is returned
- count is not destroyed
- Each call remembers the previous value

Error Handling and Debugging

Types of Error

There are three types of errors in programming

- Syntax Errors
- Runtime Errors
- Logical Errors

Syntax Errors

- Syntax errors, also called parsing errors, occur at compile time in traditional programming languages and at interpret time in JavaScript
- For example, the following line causes a syntax error because it is missing a closing parenthesis.

```
<script type = “text/javascript”>
    window.print(;
</script>
```

- When a syntax error occurs in JavaScript, only the code contained within the same thread as the syntax error is affected and the rest of the code in other threads gets executed assuming nothing in them depends on the code containing the error.

Runtime Errors

- Runtime errors, also called exceptions, occur during execution (after compilation/interpretation).
- For example, the following line causes a runtime error because here the syntax is correct, but at runtime, it is trying to call a method that does not exist.

```
<script type = “text/javascript”>
    window.printme();
</script>
```

- Exceptions also affect the thread in which they occur, allowing other JavaScript threads to continue normal execution.

Logical Errors

- Logic errors can be the most difficult type of error to track down. These errors are not the result of a syntax or runtime error. Instead, they occur when you make a mistake in the logic that drives your script and you do not get the result you expected.
- You cannot catch those error, because it depends on your business requirement what type of logic you want to put in your program.

try…catch…finally Statement

The latest versions of JavaScript added exception handling capabilities. JavaScript implements the try…catch…finally construct as well as the throw operator to handle exceptions.

You can catch programmer-generated and runtime exceptions, but you cannot catch JavaScript syntax errors.

Syntax

```
try {
    // code that may cause error
} catch (error) {
    // code to handle error
}
```

try…catch…finally Syntax

```
<script type = “text/javascript”>
        try {
                //code to run
        } catch (e) {
                // code to run if an exception occurs
        } finally {
                // code that is always executed
regardless of an exception occuring
        }
</script>
```

The **try** block must be followed by either exactly one **catch** block or one **finally** block (or one of both). When an exception occurs in the **try** block, the exception is placed in **e** and the **catch** block is executed. The optionally **finally** block executes unconditionally after try/catch.

Example: try…catch

```
<html>
<head>
        <script type = “text/javascript”>
        function myFunc() {
            try {
                let result = 10 / 0;
                console.log('Division result:', result);
                 // This will cause an error
                 undefinedFunction();
            } catch (e) {
                    console.log('Caught error:',
error);
            }
        </script>
</head>
<body>
        <p> Click the following to see the
result</p>
        <form>
        <input type=”button” value=”Click Me”
onClick=”myFunc();” />
        </form>
</body>
</html>
```

Example: With try…catch…finally

```
<html>
<head>
        <script type = “text/javascript”>
          function myFunc() {
                try {
                    console.log('📂 Opening file...');
                } catch (e) {
                    console.error('❌ Error:', error.message);
                } finally {
                    console.log('🔒 Closing file...');
                    console.log('✅ Cleanup complete');
                }
          }
        </script>
</head>
<body>
        <p> Click the following to see the
result</p>
        <form>
        <input type=”button” value=”Click Me”
onClick=”myFunc();” />
        </form>
</body>
</html>
```

Debugging

- Often, when programming code contains error, nothing will happen. There are no error messages, and you will get no indications where to search for errors. Searching for (and fixing) errors in programming code is called code debugging.
- All modern browsers have a built-in debugger.
- These debuggers provide facility to walk through the program during run-time
- With a debugger, you can set breakpoints (places where code execution can be stopped), and examine variables while the code is executing.
- It will give you the live snapshot of the program, even alter the flow of the program by forcing variable values

Console Methods

The console object in JavaScript provides various methods for debugging, logging information, and monitoring performance in the browser's developer console.

Some of the console methods are:

```
console.log("General output");

console.error("Error message");      // Red text

console.warn("Warning message");     // Yellow text

console.info("Info message");
```

Debugger Statement

The debugger statement in JavaScript is a built-in feature that acts as a programmatic breakpoint. When executed, it pauses the execution of the code and invokes any available debugging functionality.

Example:

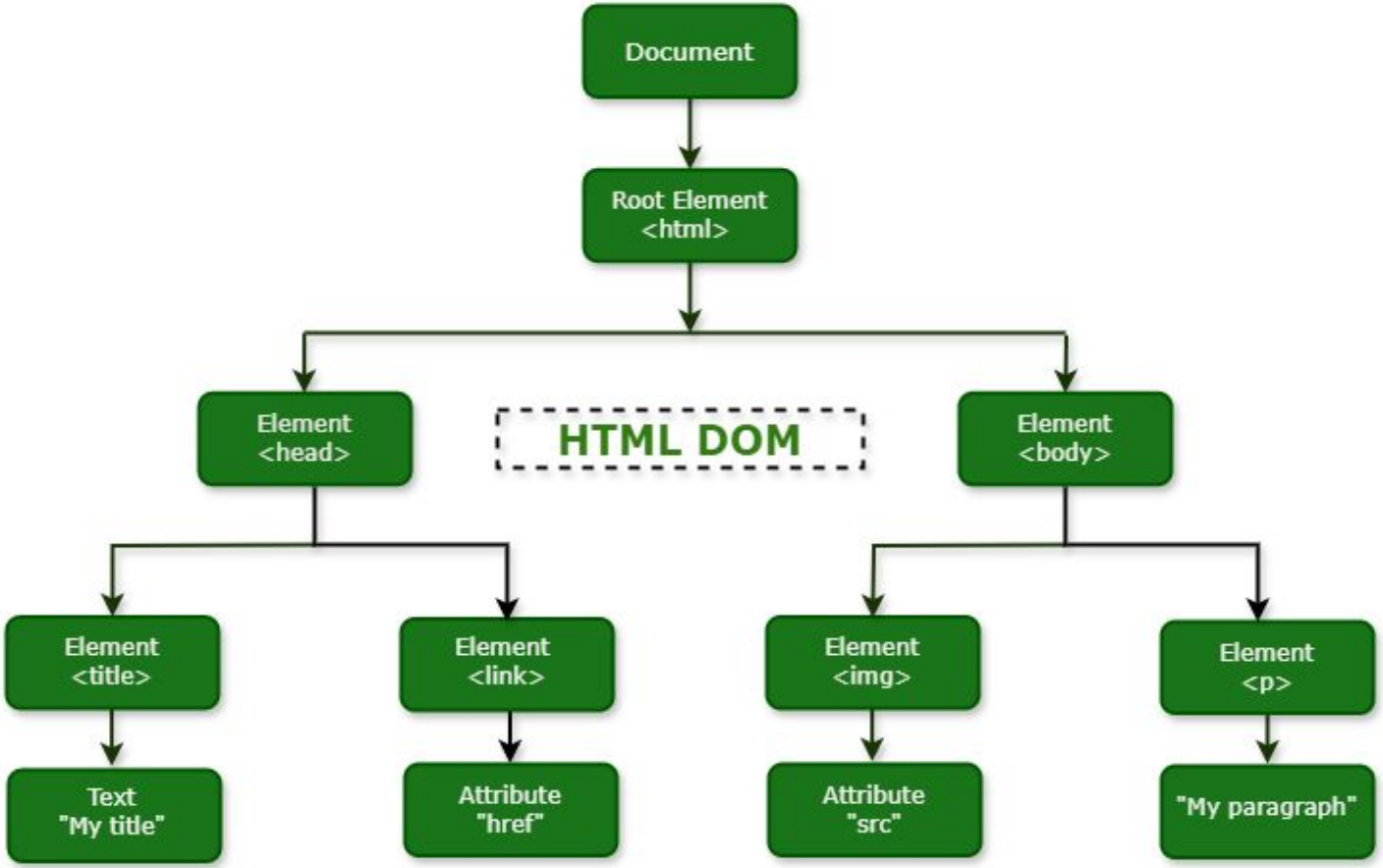
```
function calculateProduct(a, b) {
  let result = a * b;
  debugger; // Execution pauses here if dev tools are open
  return result;
}
console.log(calculateProduct(3, 4));
```

DOM Manipulation

What is the DOM?

- DOM (Document Object Model) is a programming interface that represents an HTML document as a tree of objects, where each HTML element is treated as a node.
- The DOM maps out an entire page as a document composed of a hierarchy of nodes.
- JavaScript uses the DOM to:
  - Access HTML elements
  - Modify content and styles
  - Handle user events
  - Create dynamic web pages

DOM Tree


Document
Root Element
<html>
Element
<head>
HTML DOM
Element
<body>
Element
<title>
Element
<link>
Element
<img>
Element
<p>
Text
"My title"
Attribute
"href"
Attribute
"src"
"My paragraph"

DOM == Document Object Model

```
<html>
<head>
	<title>Example JS Page</title>
</head>
<body>
	<div id=”Rayyan”>
		<ul>
			<li>Delhi</li>
			<li>Mumbai</li>
			<li>Chennai</li>
			<li>Kolkata</li>
		</ul>
	</div>
</body></html>
```

- The DOM is hierarchical

Accessing DOM Elements

- document.getElementById()
  - Returns a specific element
- document.getElementsByClassName();
  - Returns an array of elements
- document.getElementsByTagName()
  - Returns an array of elements
- document.querySelector()
  - Returns single element (first)
- document.querySelectorAll()
  - Returns NodeList

document.getElementById()

```
<body>
    <p id="score">Score: 0</p>

    <button onclick="increaseScore()">Increase Score</button>

    <script>
        var count = 0;

        function increaseScore() {
            count++;
            var scoreElement = document.getElementById("score");
            scoreElement.textContent = "Score: " + count;
        }
    </script>
</body>
```

document.getElementsByClassName()

```
<p class="fruit">Apple</p>
<p class="fruit">Banana</p>
<p class="fruit">Cherry</p>
<p class="vegetable">Carrot</p>
<p class="vegetable">Broccoli</p>

<button onclick="highlightFruits()">Highlight Fruits</button>

<script>
    function highlightFruits() {
        var fruits = document.getElementsByClassName("fruit");
        console.log(fruits);
        // getElementsByClassName returns an HTMLCollection (array-like)
        // We need to loop through it
        for (var i = 0; i < fruits.length; i++) {
            fruits[i].style.color = "red";
            fruits[i].style.fontWeight = "bold";
        }
    }
</script>
```

document.getElementsByTagName()

```
<ul>
    <li>Item One</li>
    <li>Item Two</li>
    <li>Item Three</li>
    <li>Item Four</li>
</ul>

<button onclick="colorListItems()">Color List Items</button>

<script>
    function colorListItems() {
        var listItems = document.getElementsByTagName("li");
       console.log(listItems);
        var colors = ["red", "blue", "green", "orange"];

        for (var i = 0; i < listItems.length; i++) {
            listItems[i].style.color = colors[i];
        }
    }
</script>
```

document.querySelector()

Example 1:

```
 <div id="box">This is a div with id="box"</div>

    <button onclick="selectById()">Select by
#id</button>

    <script>
        // querySelector uses CSS selector syntax

        function selectById() {
            // Select by ID using #
            var element =
document.querySelector("#box");
            element.style.backgroundColor =
"yellow";
            element.textContent = "I was selected by
#box!";
        }
    </script>
```

Example 2:

```
    <p class="message">First message</p>
     <p class="message">Second message (won't
be selected)</p>

    <button onclick="selectByClass()">Select by
.class</button>

    <script>
        function selectByClass() {
            // Select by class using . (returns FIRST
match only)
            var element =
document.querySelector(".message");
            element.style.color = "blue";
            element.textContent = "I was selected by
.message (first match only)!";
        }
    </script>
```

document.querySelectorAll()

```
<p class="note">Note 1</p>
<p class="note">Note 2</p>
<p class="note">Note 3</p>

<button onclick="highlightAll()">Highlight All Notes</button>

<script>
    function highlightAll() {
        // querySelectorAll returns ALL matching elements
        var notes = document.querySelectorAll(".note");

        // querySelectorAll returns a NodeList which supports forEach
        notes.forEach(function (note) {
            note.style.backgroundColor = "yellow";
            note.style.fontWeight = "bold";
        });
    }
</script>
```

Creating & Appending Elements

```
<div id="container"></div>

<button onclick="addParagraph()">Add Paragraph</button>

<script>
    function addParagraph() {
        // Step 1: Create a new element
        var newPara = document.createElement("p");

        // Step 2: Add content to it
        newPara.textContent = "This paragraph was created using JavaScript!";

        // Step 3: Append it to an existing element
        var container = document.getElementById("container");
        container.appendChild(newPara);
    }
</script>
```

Change Content

```
    <p id="text-example">This is plain text content.</p>
    <div id="html-example">This is <b>HTML</b> content.</div>
    <input type="text" id="input-example" value="Original value">

    <button onclick="useTextContent()">Change with textContent</button>
    <button onclick="useInnerHTML()">Change with innerHTML</button>
    <button onclick="useValue()">Change input value</button>
```

```
  <script>
      function useTextContent() {
          var element = document.getElementById("text-example");

          // textContent sets plain text (HTML tags are NOT parsed)
          element.textContent = "Changed using <b>textContent</b> - tags show as text!";
      }

      function useInnerHTML() {
          var element = document.getElementById("html-example");

          // innerHTML parses HTML tags
          element.innerHTML = "Changed using <b>innerHTML</b> - tags are <i>rendered</i>!";
      }

      function useValue() {
          var input = document.getElementById("input-example");

          // For input elements, use .value
          input.value = "New value set by JavaScript";
      }
  </script>
```

Modify Attributes

```
<img id="myImage" src="" alt="No image" width="200">

<br><br>

<a id="myLink" href="https://example.com">Visit Example</a>

<br><br>

<button onclick="changeImage()">Change Image (setAttribute)</button>

<button onclick="changeLink()">Change Link (direct property)</button>
```

```
<script>
    function changeImage() {
        var img = document.getElementById("myImage");

        // setAttribute(name, value) - sets any attribute
        img.setAttribute("src", "https://picsum.photos/200/150");
        img.setAttribute("alt", "Random image");
    }

    function changeLink() {
        var link = document.getElementById("myLink");

        // Direct property access - another way to modify attributes
        link.href = "https://google.com";
        link.textContent = "Visit Google";
        link.target = "_blank";
    }
</script>
```

Change Style

```
    <p id="demo">This is a paragraph to style.</p>


    <div id="box">This is a box</div>


    <br>

    <button onclick="changeColor()">Change Color</button>

    <button onclick="changeFont()">Change Font</button>
```

```
  <script>
      function changeColor() {
          var element = document.getElementById("demo");

          // Use element.style.propertyName
          // CSS property names become camelCase in JavaScript
          // color -> color
          // background-color -> backgroundColor
          element.style.color = "white";
          element.style.backgroundColor = "darkblue";
      }

      function changeFont() {
          var element = document.getElementById("demo");

          // font-size -> fontSize
          // font-weight -> fontWeight
          // text-align -> textAlign
          element.style.fontSize = "24px";
          element.style.fontWeight = "bold";
          element.style.fontFamily = "Arial";
      }
</script>
```

Event Handlers/Listeners

Events are things that happen in the system you are programming, which the system tells you about so your code can react to them.

Example:

- An HTML button is clicked
- A web page has finished loading
- The mouse moves over an element
- A keyboard key is pressed
- An HTML input field is changed

Syntax:

```
element.addEventListener("event", function);
```

- element → what we watch
- event → what we wait for
- function → what we do

Events

Mouse

- Click
- Dblclick
- Mousedown
- Mouseup
- Mouseover
- Mousemove
- Mouseout

Keyboard

- Keypress
- Keydown
- Keyup

Frame/Object

- Load
- Unload
- Abort
- Error
- Resize
- Scroll

Form

- Select
- Change
- Submit
- Reset
- Focus
- Blur

Mouse Event

```
<!DOCTYPE html>
<html>
<body>
<h1>JavaScript Events</h1>
<h2>The mouseover and mouseout Events</h2>

<div id="box"
style="width:200px;height:100px;padding:16px;b
order:1px solid #000;">
Move mouse over this box!
</div>
```

```
<script>
const box = document.getElementById("box");

// Let box listen for mouseover
box.addEventListener("mouseover", function () {
  box.innerHTML = "Mouse is over me!";
});

// Let box listen for mouseout
box.addEventListener("mouseout", function () {
  box.innerHTML = "Mouse is out!";
});
</script>
</body>
</html>
```

Event Listener Examples

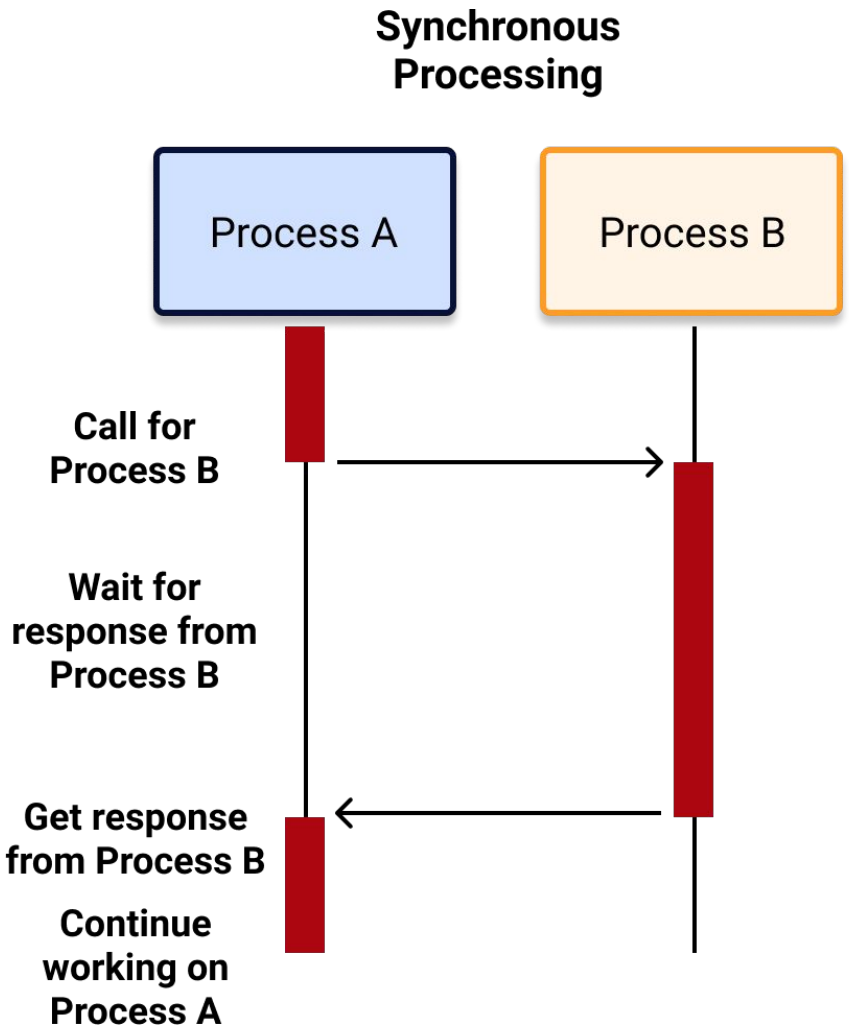
Example 1

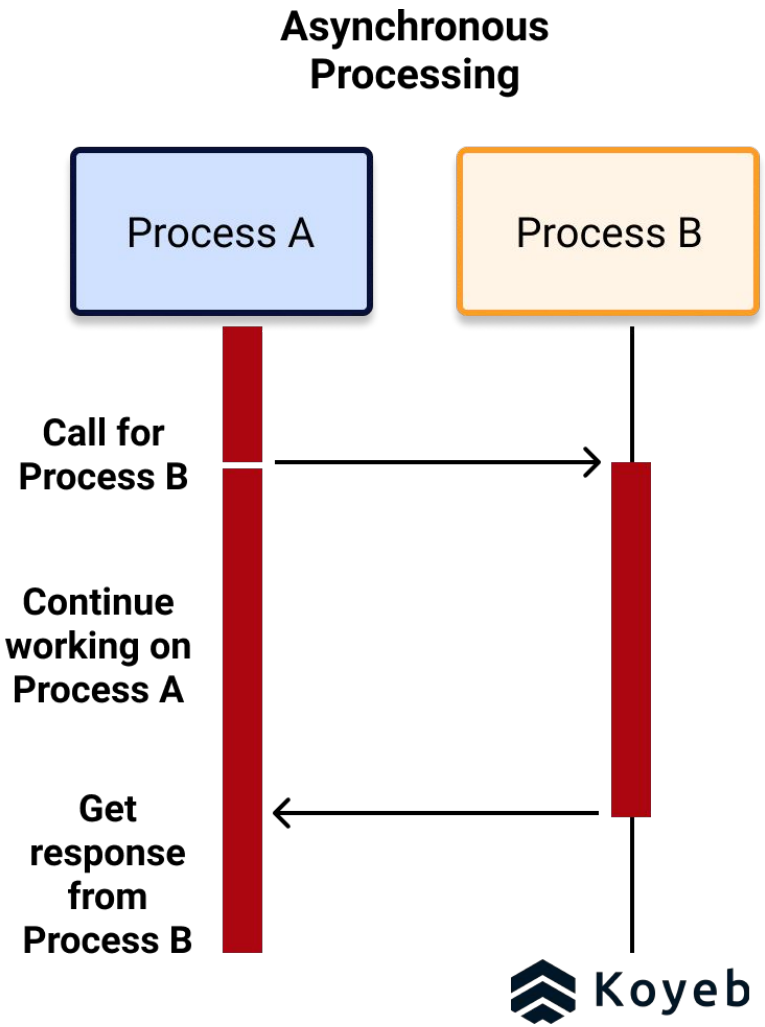
```
<button id="clickBtn">Click Me</button>
  <p id="clickCount">Clicked: 0 times</p>

  <script>
      var count = 0;
      var clickBtn =
document.getElementById("clickBtn");

      clickBtn.addEventListener("click", function
() {
          count++;

document.getElementById("clickCount").textCont
ent = "Clicked: " + count + " times";
      });
  </script>
```

Example 2

```
  <input type="text" id="nameInput" placeholder="Type
your name...">

  <script>
  <p id="preview">Preview: </p>

    // Keyup event - fires every time a key is released

      var nameInput =
document.getElementById("nameInput");

      nameInput.addEventListener("keyup", function () {

          var preview = document.getElementById("preview");

          preview.textContent = "Preview: " +
nameInput.value;

      });

  </script>
```

Asynchronous JavaScript


Synchronous Processing
Process A
Process B
Call for Process B
Wait for response from Process B
Get response from Process B
Continue working on Process A


Asynchronous Processing
Process A
Process B
Call for Process B
Continue working on Process A
Get response from Process B

Koyeb

Synchronous vs Asynchronous Execution

Synchronous

- Best suited for operations that must occur in a strict sequence
- Simpler to implement when tasks are independent
- Provides predictability, as each task completes before the next begins

Asynchronous

- Ideal for non-blocking tasks like I/O operations (eg. API requests)
- Increases efficiency when multiple tasks can run in parallel
- Improves performance and user experience

Asynchronous programming

Asynchronous programming enhances the user experience by decreasing the lag time between when a function is called and when the value of that function is returned. Async programming translates to a faster, more seamless flow in the real world.

For example, users want their apps to run fast, but fetching data from an API takes time. In these cases, asynchronous programming helps the app screen load more quickly, improving the user experience.

Why Asynchronous JavaScript?

- JavaScript is single-threaded, meaning it can execute one task at a time.

Problem with Synchronous Code

```
console.log("First Task");
console.log("Second Task");
console.log("Third Task");
```

- The program waits for each task to finish.
- Blocking operations (API calls, timers, file loading) can freeze the webpage.
- Asynchronous JavaScript allows long-running tasks to run in the background without blocking execution.

JavaScript Callbacks

A callback function is a function that is passed as an argument to another function and executed later.

- A function can accept another function as a parameter.
- Callbacks allow one function to call another at a later time.
- A callback function can execute after another function has finished

Example:

```
console.log("First Task");

setTimeout(function () { console.log("Second Task"); }, 2000);

console.log("Third Task");
```

Output: First, Third, Second (Third appears before 2 Second)

- setTimeout() is an asynchronous function that takes a callback to execute after 2 seconds.
- The rest of the code continues executing without waiting.

Callback Function Example

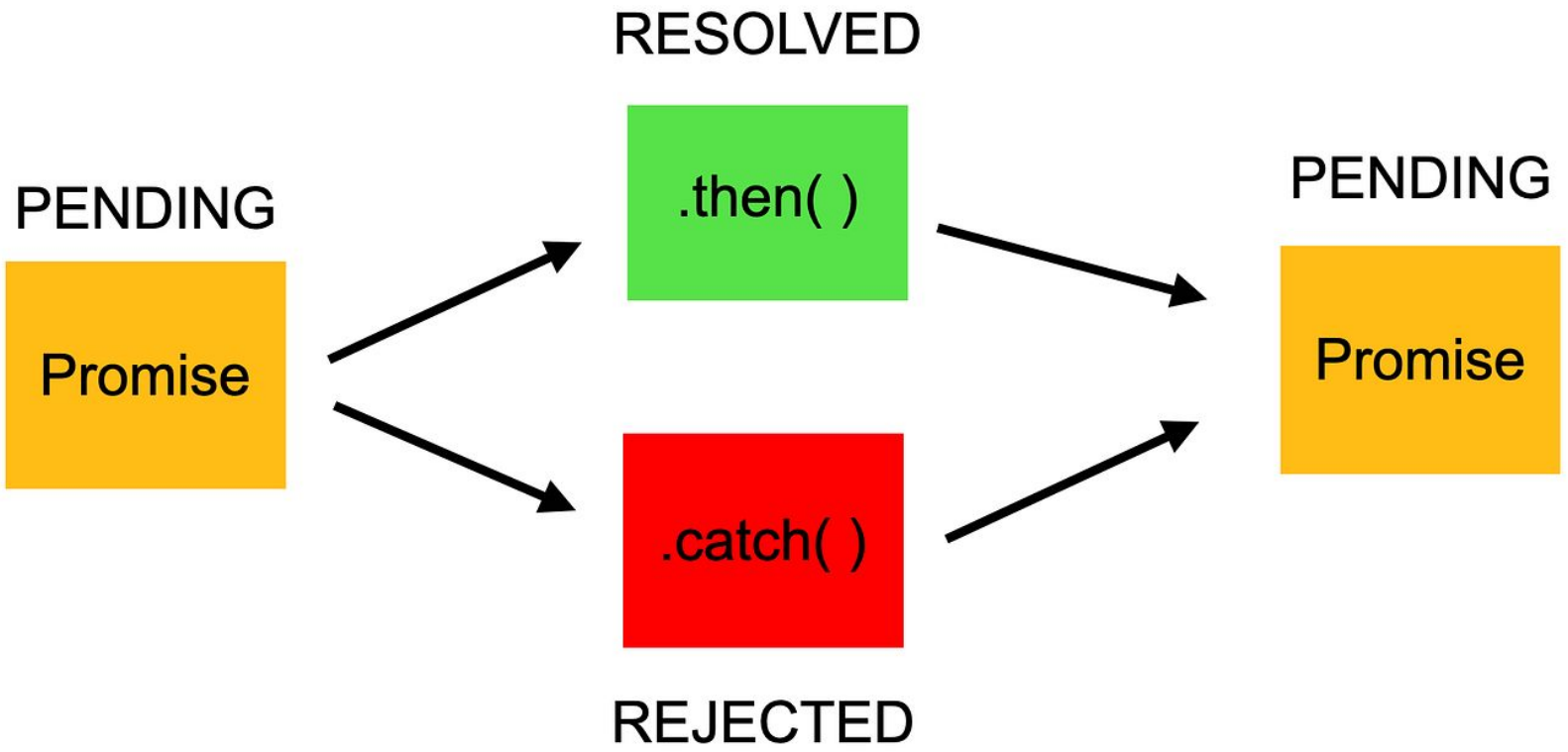
```
<p id="output"></p>

 <script>
    var output = document.getElementById("output");

    // Simple callback example
    function greet(name, callback) {
        output.innerHTML += "Hello " + name + "<br>";
        callback();
    }

    greet("Ram", function () {
        output.innerHTML += "This runs after greeting!<br>";
    });
</script>
```

Promises

A Promise is an object representing the eventual completion or failure of an asynchronous operation.

A Promise represents a value that will be:

- Pending: Initial state, neither fulfilled nor rejected
- Fulfilled: Operation completed successfully
- Rejected: Operation failed


RESOLVED
PENDING
.then( )
PENDING
Promise
Promise
.catch( )
REJECTED

Promise Simple Example

```
const isSuccess = true;

const promise = new Promise(function (resolve, reject) {
  if (isSuccess) {
    resolve("Success!");
  } else {
    reject("Failed!");
  }
});

promise
  .then(function (result) { console.log(result); })
  .catch(function (error) { console.log(error); });
```

Promise Example

```
    <p id="output"></p>
    <button onclick="startOrder()">Order Food
(Success)</button>

    <script>
        var output = document.getElementById("output");
       // Function that returns a Promise
        function orderFood(food, shouldFail) {
            return new Promise(function (resolve, reject) {
                output.innerHTML += "Ordering " + food +
"...<br>";
                setTimeout(function () {
                    if (shouldFail) {
                        reject("Sorry, " + food + " is not available!");
                    } else {
                        resolve(food);
                    }
                }, 2000);
            });
        }
```

```
// Using .then() and .catch()
        function startOrder() {
            output.innerHTML = "";
            output.innerHTML += "--- Placing Order ---<br>";

            orderFood("Momo", false)
                .then(function (message) {
                    output.innerHTML += message + "<br>";
                })
                .catch(function (error) {
                    output.innerHTML += "Error: " + error +
"<br>";
                });

            output.innerHTML += "Waiting... (this runs
first!)<br>";
        }

</script>
```

Promise Chaining Example

```
<p id="output"></p>
<button onclick="startOrder()">Order Food (Success)</button>

<script>
    var output = document.getElementById("output");
    // Function that returns a Promise
    function orderFood(food, shouldFail) {
        return new Promise(function (resolve, reject) {
            output.innerHTML += "Ordering " + food + "...<br>";
            setTimeout(function () {
                if (shouldFail) {
                    reject("Sorry, " + food + " is not available!");
                } else {
                    resolve(food + " is ready. ");
                }
            }, 2000);
        });
    }
```

```
// Using .then() and .catch()
    function startOrder() {
        output.innerHTML = "";
        output.innerHTML += "--- Placing Order ---<br>";

        orderFood("Momo", false)
            .then(function (message) {
                output.innerHTML += message + "<br>";
                // Returning a promise allows chaining
                return orderFood("Tea", false);
            })
            .then(function (message) {
                output.innerHTML += message + "<br>";
                output.innerHTML += "--- All Done! ---<br>";
            })
            .catch(function (error) {
                output.innerHTML += "Error: " + error + "<br>";
            });

        output.innerHTML += "Waiting... (this runs first!)<br>";
    }
```

async / await

- Async/await is syntactic sugar over promises, making asynchronous code look synchronous.
- Introduced in ES2017
- Built on top of Promises
- Makes async code look synchronous

Async/await Simple Example

```
function wait(isSuccess) {
        return new Promise(function (resolve,
reject) {
            setTimeout(function () {
                if (isSuccess) {
                    resolve("Success!");
                } else {
                    reject("Failed!");
                }
            }, 2000)
        });
    }
```

```
async function run() {
    console.log("Start");
    let result = await wait(true);
    console.log(result);
    console.log("End");
}

run();
```

Example: async / await

```
    <p id="output"></p>
    <button onclick="startOrder()">Order Food
(Success)</button>

    <script>
        var output = document.getElementById("output");
       // This function returns a Promise (same as
promise example)
        function orderFood(food, shouldFail) {
            return new Promise(function (resolve, reject) {
                output.innerHTML += "Ordering " + food +
"...<br>";
                setTimeout(function () {
                    if (shouldFail) {
                        reject("Sorry, " + food + " is not
available!");
                    } else {
                        resolve(food + " is ready.");
                    }
                }, 2000);
            });
        }
async function startOrder() {
            output.innerHTML = "";
            output.innerHTML += "--- Placing Order ---<br>";

            // await pauses here until the promise
resolves
            var food1 = await orderFood("Momo", false);
            output.innerHTML += food1 + " is ready!<br>";

            // This runs AFTER momo is ready
(sequential, easy to read!)
            var food2 = await orderFood("Tea", false);
            output.innerHTML += food2 + " is ready!<br>";

            output.innerHTML += "--- All Done! ---<br>";
        }
<
```

JSON & AJAX

What is JSON?

- A lightweight text based data-interchange format
- Completely language independent
- Based on a subset of the JavaScript Programming Language
- Easy to understand, manipulate and generate

Definition:

- JSON (JavaScript Object Notation) is a text-based data format that represents structured data using key–value pairs and is easy for both humans and machines to read and write.

JSON Syntax Rules

- Data is in name/value pairs
- Data is separated by commas
- Curly braces {} hold objects
- Square brackets [] hold arrays
- Strings must be in double quotes
- Values can be:
  - string
  - number
  - boolean
  - array
  - object
  - null

JSON Example

```
{
    "name": "John Doe",
    "age": 30,
    "isStudent": false,
    "courses": ["JavaScript", "HTML", "CSS"],
    "address": {
        "city": "Kathmandu",
        "country": "Nepal"
    },
    "spouse": null
}
```

Working with JSON in JavaScript: JSON.stringify()

Convert JavaScript object to JSON string.

Example:

```
const user = {
    name: "John",
    age: 30,
    hobbies: ["reading", "gaming"],
    isActive: true
};
const jsonString = JSON.stringify(user);
console.log(jsonString);
```

Working with JSON in JavaScript: JSON.parse()

Convert JSON string to JavaScript object.

Example

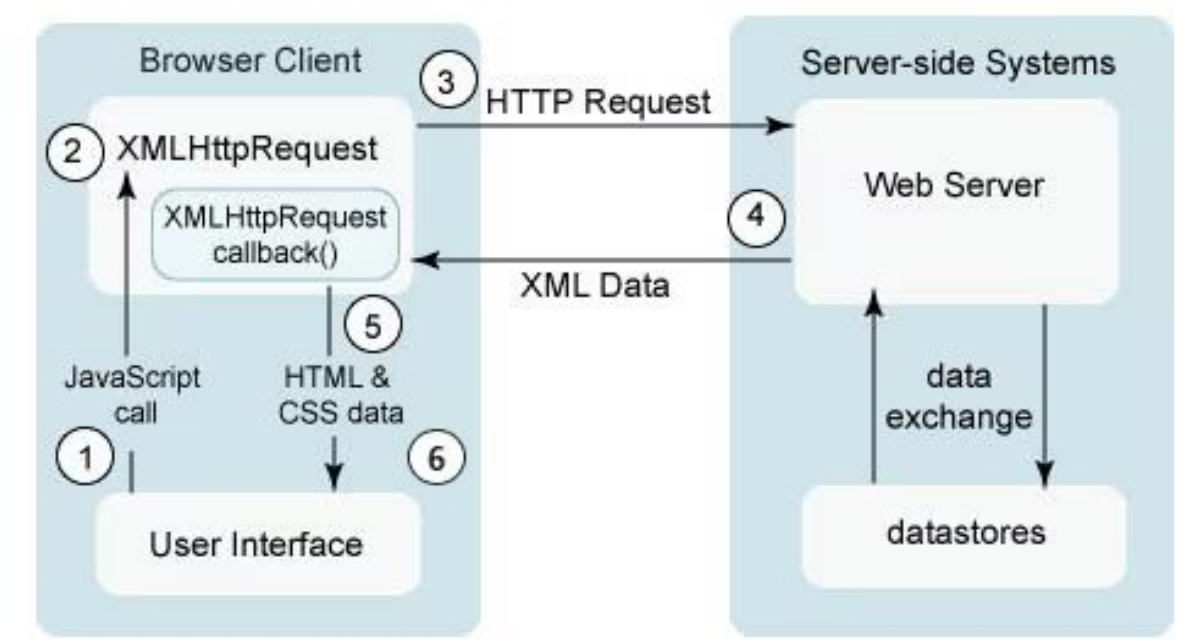
```
const jsonString = '{"name":"John","age":30,"isActive":true}';


const user = JSON.parse(jsonString);

console.log(user.name);  // "John"

console.log(user.age);   // 30
```

What is AJAX?

- AJAX (Asynchronous JavaScript and XML) is a technique that allows web pages to communicate with a server asynchronously, without reloading the page.
- AJAX enables asynchronous data exchange between client and server, allowing dynamic updating of web content.

How AJAX Works

1. User triggers an event
2. Browser sends request to server
3. Server processes request
4. Server sends data (JSON)
5. Webpage updates dynamically



AJAX Using XMLHttpRequest

- XMLHttpRequest (XHR) is the traditional JavaScript object used to:
  - Send HTTP requests
  - Receive responses from a server
  - Update webpage content without page reload
- XMLHttpRequest is a JavaScript object that enables asynchronous communication between the client and server.

AJAX GET Request Using XMLHttpRequest

```
<button onclick="fetchPost()">Fetch Post </button>
<p id="output"></p>
<script>
   var output = document.getElementById("output");

  function fetchPost() {
          const xhr = new XMLHttpRequest();
         xhr.open("GET",
"https://jsonplaceholder.typicode.com/posts/1", true);

     xhr.onload = function () {
        if (xhr.status === 200) {
           // Parse the JSON response
          var post = JSON.parse(xhr.responseText);

          output.innerHTML += "Status: " + xhr.status + "
OK<br><br>";
          output.innerHTML += "--- Post Details ---<br>";
          output.innerHTML += "Title: " + post.title + "<br>";
          output.innerHTML += "Body: " + post.body + "<br>";
             }
        else {
           output.innerHTML += "Error! Status: " +
xhr.status + "<br>";
              }
        };

        // Handle network errors
          xhr.onerror = function () {
             output.innerHTML += "Network error
occurred!<br>";
          };

          // Send the request
          xhr.send();
}
```

AJAX POST Request Using XMLHttpRequest

```
  <button onclick="sendData()">Send Data
</button>
<p id="output"></p>
<script>

var output = document.getElementById("output");

function sendData() {
        const xhr = new XMLHttpRequest();
         xhr.open("POST",
"https://jsonplaceholder.typicode.com/posts", true);

        xhr.setRequestHeader("Content-Type",
"application/json");
```

```
xhr.onload = function () {
  if (xhr.status === 201) {
var response = JSON.parse(xhr.responseText);
        //Server Response
       output.innerHTML += "ID: " + response.id +
"<br>";
       output.innerHTML += "Title: " + response.title +
"<br>";
       output.innerHTML += "Body: " + response.body
+ "<br>";
             }
    };
 // Create data to send as JSON
          var data = {
             title: "Hello from JavaScript",
             body: "This is a test post from AJAX",
             userId: 1
          };

// Convert object to JSON string and send
          xhr.send(JSON.stringify(data));
}
```

AJAX GET Request Using Fetch API

```
fetch("https://example.com/api/test", {
    method: "GET"
})
.then(function (response) {
    return response.json();
})
.then(function (data) {
    console.log(data);
})
.catch(function (error) {
    console.log("Error:", error);
});
```

- fetch() sends an asynchronous HTTP GET request to the given URL
- method: "GET" specifies the request type
- response.json() converts the JSON response into a JavaScript object
- .then() handles the successful response
- .catch() handles errors
- The retrieved JSON data is logged in the browser console

AJAX GET Request Using Fetch API

```
<button onclick="fetchUser()">Fetch User</button>
<p id="output"></p>
<script>
var output = document.getElementById("output");

  function fetchUser() {

        fetch("https://jsonplaceholder.typicode.com/users/1", { method: GET })
            .then(function (response) {
                output.innerHTML += "Response received! Status: " + response.status + "<br>";
                // response.json() also returns a Promise
                return response.json();
            })
        .then(function (data) {
                // data is now a JavaScript object (parsed from JSON)
                output.innerHTML += "<br>--- User Details ---<br>";
                output.innerHTML += "Name: " + data.name + "<br>";
                output.innerHTML += "Email: " + data.email + "<br>";
                output.innerHTML += "City: " + data.address.city + "<br>";
                output.innerHTML += "Company: " + data.company.name + "<br>";
            })
        .catch(function (error) {
                output.innerHTML += "Error: " + error.message + "<br>";
            });
}
```

AJAX POST Request Using Fetch API

```
<button onclick="addUser()">Add User</button>
<p id="output"></p>
<script>
  var output = document.getElementById("output");

function addUser() {
        // Create a new user object to send
        var newUser = {
              name: "John Doe",
              username: "john123",
              email: "john@example.com",
              phone: "9841000000",
              website: "john.com.np"
         };
```

```
fetch("https://jsonplaceholder.typicode.com/users", {
              method: "POST",
              headers: {
                 "Content-Type": "application/json"
              },
              body: JSON.stringify(newUser)
      })
      .then(function (response) {
            output.innerHTML += "Response Status: " + response.status + "<br>";
            return response.json();
       })
      .then(function (data) {
            // Server returns the created user with an id
            output.innerHTML += "User Added Successfully<br>";
            output.innerHTML += "ID (from server): " + data.id + "<br>";
            output.innerHTML += "Name: " + data.name + "<br>";
            output.innerHTML += "Username: " + data.username + "<br>";
            output.innerHTML += "Email: " + data.email + "<br>";
            output.innerHTML += "Phone: " + data.phone + "<br>";
       })
       .catch(function (error) {
             output.innerHTML += "Error: " + error.message + "<br>";
        });
}
```

ES6 and Modern JavaScript

Introduction to ES6

ES6 (ECMAScript 2015) is a major update to JavaScript that introduced new syntax and features to make code:

- More readable
- More secure
- Easier to maintain
- Suitable for large-scale applications

let, const and var

var

- Function scoped
- Can be redeclared
- Hoisted

Example:

var x = 10;

var x = 20;

let

- Block scoped
- Cannot be redeclared
- Mutable

Example:

let y = 10;

y = 15;

const

- Block scoped
- Cannot be redeclared or reassigned
- Used for constants

Example:

const PI = 3.14;

Arrow Functions

Arrow functions provide shorter syntax and lexical this binding.

Traditional

```
function greet() {
    console.log("Hello");
}
```

Arrow

```
const greet = () => console.log("Hello");
```

Template Literals

Template literals allow string interpolation using backticks ( ).

```
let name = "Alice";

let msg = `Welcome ${name} to JavaScript`;
```

Destructuring Assignment

Extract values from arrays or objects easily.

Array Destructuring

let [a, b] = [10, 20];

Object Destructuring

let student = { name: "Bob", age: 22 };

let { name, age } = student;

Default Parameters

```
function greet(name = "Guest") {
    console.log("Hello " + name);
}
```

Spread and Rest Operators (...)

Spread Operator

```
let arr1 = [1, 2];

let arr2 = [...arr1, 3, 4];
```

Rest Operator

```
function sum(...nums) {
    return nums.reduce((a, b) => a + b);
}
```

JavaScript Modules (import / export)

Importing

```
import { add } from "./math.js";


console.log(sum(5, 5));
```

Exporting

```
export function add(a, b) {
    return a + b;
}


export const PI = 3.14;
```

Modules allow JavaScript code to be organized into reusable files using import and export.

JavaScript Libraries

Introduction to JavaScript Libraries

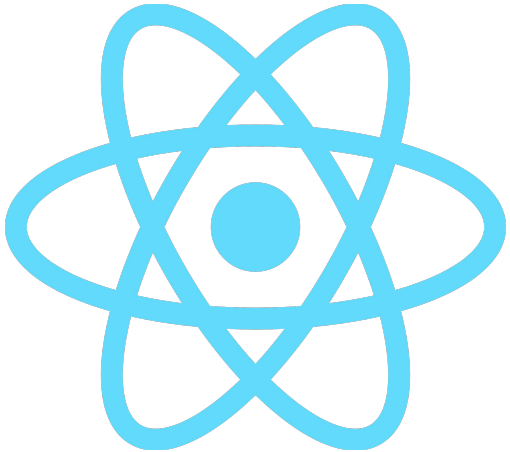
- A JavaScript library is a collection of pre-written JavaScript code that helps developers perform common tasks more efficiently.
- A JavaScript library is a reusable collection of functions and objects that simplifies complex JavaScript tasks such as DOM manipulation, event handling, and AJAX operations.

Libraries reduce:

- Code length
- Development time
- Programming complexity

Vue.js

Angular



React

Why Use JavaScript Libraries?

- Faster development
- Cross-browser compatibility
- Better performance
- Cleaner and maintainable code
- Industry-standard practices

Library vs Framework

Feature	Library	Framework
Control	Developer controls flow	Framework controls flow
Flexibility	High	Less
Usage	Specific tasks	Full application
Example	React	Angular

React.js

React is a JavaScript library developed by Facebook for building user interfaces, especially Single Page Applications (SPA).

Key Features

- Component-based architecture
- Virtual DOM for fast updates
- Reusable UI components
- Uses JSX syntax
- Large ecosystem (React Router, Redux)

Angular

Angular is a full-fledged JavaScript framework developed by Google.

Key Features

- MVC architecture
- Two-way data binding
- TypeScript based
- Built-in routing and services

Vue.js

Vue.js is a progressive JavaScript framework used for building user interfaces.

Key Features

- Easy to learn
- Two-way data binding
- Component-based
- Lightweight

Any Questions?